

Word Embedding

Converting Words into Useful Numbers

CSE427: Machine Learning

Why Do We Need Word Embeddings?

Neural networks work with numbers, not raw words.

word $\rightarrow$ number/vector

Bad idea: assign random numbers.

great = 4.2, awesome = -32.1

Problem:

great and awesome are similar, but their numbers are very different.

Better idea:

Similar words should have similar numerical representations.

Example:

great $\approx$ awesome

bad $\approx$ terrible

What Is a Word Embedding?

A word embedding is a vector representation of a word.

$$\text{word} \rightarrow [e_1, e_2, \dots, e_d]$$

Example with two embedding dimensions:

$$\text{great} \rightarrow [1.85, 0.98]$$

$$\text{awesome} \rightarrow [1.81, 0.99]$$

Since these vectors are close, the model can understand that the words are used similarly.

Word embedding = meaningful numbers learned from context.

Context Matters

The same word can be used in different contexts.

Positive use:

"This book is great!"

Sarcastic or negative use:

"My phone is broken, great."

So one number may not be enough.

A word can have multiple embedding dimensions:

great $\rightarrow$ [1.85, 0.98]

One dimension may capture positive usage, another may help capture different context.

More embedding dimensions allow richer meaning.

Training Data Example

Suppose the training data has two sentences:

MovieA is great!

MovieB is great!

Unique words:

MovieA, MovieB, is, great

Vocabulary size:

$$V = 4$$

If we want 2 embedding values per word:

$$d = 2$$

Then the embedding table has size:

$$V \times d = 4 \times 2$$

One-Hot Encoding

Before learning embeddings, words are represented as one-hot vectors.

Vocabulary:

[MovieA, is, great, MovieB]

One-hot vectors:

MovieA = [1, 0, 0, 0]

is = [0, 1, 0, 0]

great = [0, 0, 1, 0]

MovieB = [0, 0, 0, 1]

One word gets 1, all other positions get 0.

Embedding Layer Idea

The embedding layer maps a one-hot word vector to a dense vector.

one-hot word $\rightarrow$ embedding vector

Example:

$$[1, 0, 0, 0] \rightarrow [0.38, 0.42]$$

The embedding values are weights learned by the neural network.

Embedding values are trainable weights.

If the vocabulary has V words and embedding dimension is d , then:

Embedding matrix size = $V \times d$

How Are Embeddings Learned?

We train the network using a prediction task.

For the sentence:

MovieA is great

we can train:

MovieA $\rightarrow$ is

is $\rightarrow$ great

For the sentence:

MovieB is great

we can train:

MovieB $\rightarrow$ is

Because MovieA and MovieB appear in similar contexts, training should make their embeddings similar.

Softmax Prediction

After the embedding layer, the network predicts the next word.

Output scores:

$$z_1, z_2, \dots, z_V$$

Softmax converts scores into probabilities:

$$P(y = k) = \frac{e^{z_k}}{\sum_{j=1}^V e^{z_j}}$$

Example output:

$$P(\text{MovieA}) = 0.23$$

$$P(\text{is}) = 0.27$$

$$P(\text{great}) = 0.26$$

$$P(\text{MovieB}) = 0.24$$

Training Objective

If the input is:

MovieA

and the correct next word is:

is

then we want:

$$P(\text{is}) \approx 1$$

and other probabilities close to 0.

Loss function:

$$L = - \sum_{k=1}^V y_k \log(\hat{y}_k)$$

This is cross-entropy loss.

Backpropagation updates:

Visualizing Embeddings

If each word has 2 embedding values:

$$\text{word} \rightarrow [e_1, e_2]$$

we can plot:

$$x\text{-axis} = e_1$$

$$y\text{-axis} = e_2$$

Before training, similar words may be far apart.

After training:

MovieA and MovieB appear in similar contexts

so their embeddings become closer.

Embedding space captures similarity.

Important Point: We Care About the Weights

In ordinary neural networks, we usually care about the final output.

In word embedding networks, the final output is mainly used for training.

The useful result is:

the learned embedding weights

These weights become the word vectors.

Continuous Bag of Words

Continuous Bag of Words, or CBOW, predicts the middle word from surrounding words.

Example sentence:

MovieA is great

Use surrounding words:

MovieA, great

to predict the middle word:

is

CBOW: context words → center word

Skip-Gram

Skip-Gram does the opposite of CBOW.

It uses the middle word to predict surrounding words.

Example:

MovieA is great

Use center word:

is

to predict context words:

MovieA, great

Skip-Gram: center word → context words

CBOW vs Skip-Gram

Method	Input	Target
CBOW	surrounding words	center word
Skip-Gram	center word	surrounding words

Example sentence:

MovieA is great

CBOW:

(MovieA, great) $\rightarrow$ is

Skip-Gram:

is $\rightarrow$ (MovieA, great)

Both methods learn embeddings from context.

Large Vocabulary Problem

In practice, vocabulary can be very large:

$$V > 10,000$$

The embedding dimension may be hundreds:

$$d = 100, 300, 500$$

Training with full softmax over all words can be expensive.

For every training example, softmax considers:

all vocabulary words

This is slow for large vocabulary.

Large vocabulary makes training expensive.

Negative Sampling

Negative sampling speeds up training.

Instead of updating all output words, we update:

the correct word

and only a few randomly selected incorrect words.

Example:

aardvark $\rightarrow$ eats

Correct output:

eats = 1

Sampled wrong outputs:

banana = 0, car = 0, river = 0

So the model does not need to update every word in the vocabulary.

Negative sampling reduces computation.

Final Summary

- Neural networks need numbers, not raw words.
- Word embeddings convert words into dense vectors.
- Similar words should have similar vectors.
- Embeddings are learned weights.
- One-hot vectors select rows from the embedding matrix.
- The network can be trained by predicting nearby words.
- CBOW predicts a center word from context.
- Skip-Gram predicts context words from a center word.
- Negative sampling makes training faster for large vocabularies.

Word embeddings learn meaning from context.